

C2SafeRust-LLM

Module 5: Ownership Inference & Code Generation -- Line-by-Line Explanation

For Beginners | Mini Project S3

Module 5: Ownership Inference & Rust Code Generation

Line-by-Line Code Explanation for Beginners

Files: `src/module5_codegen/mod.rs`, `ownership_infer.rs`, `rust_codegen.rs`

What does Module 5 do? Module 5 is where the "thinking" becomes "writing." It takes everything learned by Modules 3 and 4 and makes two big decisions: STEP 5.1 — Ownership Inference (`ownership_infer.rs`): For every variable in every function, decide WHICH Rust ownership type it needs: `Box<T>?` `&mut T?` `&T?` `Vec<T>?` `Option<Box<T>>?` STEP 5.2 — Rust Code Synthesis (`rust_codegen.rs`): Use the ownership decisions to generate actual syntactically valid Rust source code. Translate C structs → Rust structs, C unions → Rust enums, C functions → Rust functions. The output is a String of Rust source code written to a file like `output/output.rs`.

```
D1: HashMap<String, SemanticModel>    (from Module 3)
D2: InterproceduralSummaries          (from Module 4)
TranslationUnitAST                    (from Module 2)
    |
    | Step 5.1: ownership_infer.rs
    v
HashMap<String, FunctionOwnershipMap> (one entry per function)
    |
    | Step 5.2: rust_codegen.rs
    v
String (Rust source code) --> written to output.rs
```

File 1: `src/module5_codegen/mod.rs`

(The Orchestrator — Infer, Synthesize, Write)

Lines 1-13: Imports and declarations

```

pub mod ownership_infer;
pub mod rust_codegen;

pub use ownership_infer::{infer_ownership_types, FunctionOwnershipMap};
pub use rust_codegen::synthesize_rust_code;

use std::fs;
use std::path::Path;
use crate::module2_ast::TranslationUnitAST;
use crate::data_stores::semantic_models::SemanticModel;
use crate::data_stores::interprocedural_summaries::InterproceduralSummaries;

```

Import	Why
<code>`std::fs`</code>	For <code>`fs::write()`</code> — write the generated Rust code to a file.
<code>`std::path::Path`</code>	The output file path parameter type.
<code>`InterproceduralSummaries`</code>	D2 — needed by both ownership inferer and codegen.

Lines 14-19: The Output Struct `CodegenResult`

```

#[derive(Debug, Clone)]
pub struct CodegenResult {
    pub generated_rust_code: String,
    pub output_file_path: String,
    pub ownership_maps: HashMap<String, FunctionOwnershipMap>,
}

```

Field	What it holds
<code>`generated_rust_code`</code>	The ENTIRE generated Rust source code as one big String — returned to Module 6 for compiler validation.
<code>`output_file_path`</code>	The file where the code was written (e.g., <code>"output/output.rs"</code>). Shown in the GUI.
<code>`ownership_maps`</code>	The intermediate ownership decisions. Passed to Module 7 for the report.

Lines 21-43: The Main Entry Point `generate_rust_code()`

```
pub fn generate_rust_code(
    tu_ast: &TranslationUnitAST,
    semantic_models: &HashMap<String, SemanticModel>,
    summaries: &InterproceduralSummaries,
    output_path: &Path,
) -> Result<CodegenResult, String> {
    // Step 5.1: Ownership Classification
    let ownership_maps = infer_ownership_types(tu_ast, semantic_models, summaries);

    // Step 5.2: Rust Code Synthesis
    let generated_rust_code = synthesize_rust_code(tu_ast, &ownership_maps, summaries);

    // Write to file
    fs::write(output_path, &generated_rust_code)
        .map_err(|err| format!(
            "Failed to write candidate Rust output to {:?}: {}", output_path, err
        ))?;

    Ok(CodegenResult {
        generated_rust_code,
        output_file_path: output_path.to_string_lossy().to_string(),
        ownership_maps,
    })
}
```

	Code	Explanation		
	Borrow path	Where to save the generated <code>.rs` file.</code>		
<code>(tu_ast, summaries)`</code>	Step 5.1	Returns <code>`HashMap`</code> . Uses D1 + D2 + AST.		
<code>tu_ast, summaries)`</code>	Step 5.2	Returns <code>`String`</code> — the generated Rust source code.		
<code>,`</code>	Write to disk	Writes the Rust string to a <code>.rs` file atomically. Returns <code>`Result<(), io::Error>`</code>.</code>		
	<code>err\`</code>	<code>format!(...)`</code>	Error conversion	Converts the description.
	Error propagation	If writing fails, return <code>`Err(String)`</code> immediately.		
<code>_lossy().`</code>	Path → String	Converts the path to a human-readable String for the output struct. <code>`to_string_lossy()`</code> handles non-UTF-8 paths (Windows paths with special chars).		

Why write to disk here (Module 5) and not later? Module 6's compiler validation loop needs a **PHYSICAL FILE** to run `rustc` on. `rustc` is a command-line tool — it reads files, not in-memory strings. So Module 5 must write the output before Module 6 can validate it.

File 2: `src/module5_codegen/ownership_infer.rs`

(Classifying Every Variable's Rust Ownership Type)

The Core Problem: In C, ALL pointers look the same: `int*`, `int**`, etc. In Rust, they have VERY different types with different ownership semantics: C `int*` could become: `&T` -- shared borrow (read-only reference) `&mut T` -- mutable borrow (can write through) `Box<T>` -- owned heap pointer (unique owner) `Vec<T>` -- owned heap array C `int**` with `*out = malloc(n)` MUST become something that represents the CREATION of heap memory. The ownership inferer reads D1 (memory patterns, pointer depths) and D2 (alias verdicts, ownership transfer summaries, data dep flags) to make this classification decision.

Lines 1-5: Imports

```
use std::collections::HashMap;
use crate::module2_ast::TranslationUnitAST;
use crate::data_stores::semantic_models::{MemoryPattern, PointerDepthClassification, SemanticM
use crate::data_stores::interprocedural_summaries::{
    InterproceduralSummaries, OwnershipKind, OwnershipTransferType
};
```

Import	Why
<code>`MemoryPattern`</code>	<code>`Free`</code> , <code>`Allocate`</code> , <code>`Write`</code> , <code>`Read`</code> — from D1.
<code>`PointerDepthClassification`</code>	<code>`SingleLevel`</code> , <code>`MultiLevel{depth}`</code> — from D1.
<code>`OwnershipKind`</code>	The OUTPUT: <code>`UniqueHeapOwner`</code> , <code>`MutableBorrow`</code> , <code>`SharedBorrow`</code> , <code>`HeapArray`</code> , <code>`ConservativeFallback`</code> .
<code>`OwnershipTransferType`</code>	<code>`ConsumesOwnership`</code> , <code>`CreatesOwnership`</code> , <code>`None`</code> — from D2.

Lines 7-13: The Per-Variable Ownership Record

```
#[derive(Debug, Clone)]
pub struct VariableOwnership {
    pub variable_name: String,
    pub inferred_kind: OwnershipKind,
    pub rust_type_hint: String,
}
```

Field	Example	What it stores
<code>`variable_name`</code>	<code>`"ptr"`</code>	The C variable name.
<code>`inferred_kind`</code>	<code>`OwnershipKind::UniqueHeapOwner`</code>	The enum variant representing the ownership category.
<code>`rust_type_hint`</code>	<code>`"Box"`</code>	A STRING representation of the recommended Rust type. Used by the codegen.

Lines 15-18: The Per-Function Ownership Collection

```
#[derive(Debug, Clone)]
pub struct FunctionOwnershipMap {
    pub per_variable: Vec<VariableOwnership>,
}
```

- One `FunctionOwnershipMap` per function.
- `per_variable` holds the `VariableOwnership` for every variable in that function.
- The outer `HashMap<String, FunctionOwnershipMap>` maps function names to their maps.

Lines 20-55: The Main Inference Function `infer_ownership_types()`

```
pub fn infer_ownership_types(
    tu_ast: &TranslationUnitAST,
    semantic_models: &HashMap<String, SemanticModel>,
    summaries: &InterproceduralSummaries,
) -> HashMap<String, FunctionOwnershipMap> {
    let mut result = HashMap::new();

    for func in &tu_ast.functions {
        if let Some(model) = semantic_models.get(&func.name) {
            let mut per_variable = Vec::new();

            for (var_name, pattern) in &model.memory_patterns {
                let ownership_kind = infer_single_variable(
                    func_name, var_name, pattern, model, summaries
                );
                let rust_type_hint = ownership_kind_to_rust_type(&ownership_kind, var_name, model);
                per_variable.push(VariableOwnership {
                    variable_name: var_name.clone(),
                    inferred_kind: ownership_kind,
                    rust_type_hint,
                });
            }

            result.insert(func.name.clone(), FunctionOwnershipMap { per_variable });
        }
    }
    result
}
```

Line	Code	Explanation
<code>`for func in &tu_ast.functions`</code>	Loop over functions	Process every function in the program.
<code>`for (var_name, pattern) in &model.memory_patterns`</code>	Loop over variables	D1's <code>`memory_patterns`</code> has every variable we tracked.
<code>`infer_single_variable(...)`</code>	Core decision	Private helper that applies the 5-rule decision tree (see below).

Line	Code	Explanation
<code>`ownership_kind_to_rust_type(&ownership;_kind, ...)`</code>	String conversion	Converts the enum variant to a Rust type string like <code>"Box"</code> .
<code>`result.insert(func.name.clone(), FunctionOwnershipMap { per_variable })`</code>	Store result	One map per function stored in the output HashMap.

The Core Decision Function `infer_single_variable()` *(Lines 57-92)*

```

fn infer_single_variable(
    func_name: &str,
    var_name: &str,
    pattern: &MemoryPattern,
    model: &SemanticModel,
    summaries: &InterproceduralSummaries,
) -> OwnershipKind {
    // Rule 1: Data-Dependent Ownership -> ConservativeFallback (Idiom 6)
    let is_data_dep = summaries.data_dependent_ownership_flags
        .iter()
        .any(|f| f.function_name == func_name && f.variable_name == var_name);
    if is_data_dep {
        return OwnershipKind::ConservativeFallback;
    }

    // Rule 2: Ownership Transfer T** -> UniqueHeapOwner (Idiom 2 & 3)
    let transfer_key = format!("{:?}", func_name, var_name);
    if let Some(summary) = summaries.ownership_transfer_summaries.get(&transfer_key) {
        match &summary.transfer_type {
            OwnershipTransferType::ConsumesOwnership |
            OwnershipTransferType::CreatesOwnership => {
                return OwnershipKind::UniqueHeapOwner;
            }
            OwnershipTransferType::None => {}
        }
    }

    // Rule 3: Heap Allocation -> UniqueHeapOwner
    if matches!(pattern, MemoryPattern::Allocate) {
        let depth_class = model.pointer_depth_classifications.get(var_name);
        if matches!(depth_class, Some(PointerDepthClassification::MultiLevel { .. })) {
            return OwnershipKind::HeapArray; // T** + malloc = array
        }
        return OwnershipKind::UniqueHeapOwner; // T* + malloc = single object
    }

    // Rule 4: Write through pointer -> MutableBorrow
    if matches!(pattern, MemoryPattern::Write) {
        return OwnershipKind::MutableBorrow;
    }

    // Rule 5: Default -> SharedBorrow (read-only)
    OwnershipKind::SharedBorrow
}

```

The 5-Rule Decision Tree (in priority order):

Priority	Rule	Condition Checked	Result	Why
1 (highest)	Data-Dependent	Var is in `data_dependent_ownership_flags` (D2)	`ConservativeFallback` → `Option`	Idiom 6: ownership can't be statically determined

Priority	Rule	Condition Checked	Result	Why
2	Ownership Transfer	Var has entry in `ownership_transfer_summaries` (D2) with Consume or Create	`UniqueHeapOwner` → `Box`	Idiom 2 or 3: T** moves heap ownership
3a	Heap Allocation + MultiLevel	`pattern == Allocate` AND `pointer_depth >= 2`	`HeapArray` → `Vec`	T** + malloc usually means dynamic array
3b	Heap Allocation + SingleLevel	`pattern == Allocate` AND `pointer_depth < 2`	`UniqueHeapOwner` → `Box`	T* + malloc = single owned heap object
4	Write	`pattern == Write`	`MutableBorrow` → `&mut; T`	Writing through pointer = mutable reference
5 (default)	Read or Free	Everything else	`SharedBorrow` → `&T;`	Read-only access = shared immutable reference

Why is Rule 1 checked FIRST? Idiom 6 (data-dependent ownership) is the MOST conservative case. Even if a variable also has `Allocate` or `Write` patterns, if its ownership is runtime-conditional, we MUST fall back to `Option<Box<T>>` regardless. The priority order is critical for correctness.

Understanding Rule 1 — `data_dependent_ownership_flags` search:

```
summaries.data_dependent_ownership_flags
    .iter()
    .any(|f| f.function_name == func_name && f.variable_name == var_name)
```

- `.iter()` — iterate through the Vec of `DataDepFlags`.
- `.any(|f| ...)` — returns `true` if ANY flag matches BOTH the function name AND variable name.
- This is more precise than a HashMap lookup — we check both dimensions.

Understanding Rule 2 — ownership transfer lookup:

```
let transfer_key = format!("{:?}", func_name, var_name);
summaries.ownership_transfer_summaries.get(&transfer_key)
```

- Key format `"free_buffer:buf"` matches exactly what Module 4 stored.
 - `match &summary.transfer_type { ConsumesOwnership | CreatesOwnership => ... }`
- Rust allows matching multiple patterns with `|` in a single arm.

The Type Hint Generator `ownership_kind_to_rust_type()` (Lines 94-130)

```
fn ownership_kind_to_rust_type(
    kind: &OwnershipKind,
    var_name: &str,
    model: &SemanticModel,
) -> String {
    let base_type = get_base_c_type(var_name, model);
    let rust_base = c_type_to_rust(&base_type);

    match kind {
        OwnershipKind::UniqueHeapOwner      => format!("Box<{}>", rust_base),
        OwnershipKind::MutableBorrow         => format!("&mut {}", rust_base),
        OwnershipKind::SharedBorrow          => format!("&{}", rust_base),
        OwnershipKind::HeapArray              => format!("Vec<{}>", rust_base),
        OwnershipKind::ConservativeFallback => format!("Option<Box<{}>>", rust_base),
    }
}
```

`get_base_c_type()` — look up C type from symbol table:

```
fn get_base_c_type(var_name: &str, model: &SemanticModel) -> String {
    model.symbol_table
        .get(var_name)
        .cloned()
        .unwrap_or_else(|| "i32".to_string())
}
```

- `.get(var_name)` returns `Option<&String>`.
- `.cloned()` converts `Option<&String>` to `Option<String>` (makes a copy).
- `.unwrap_or_else(|| "i32".to_string())` — if the variable isn't in the symbol table, default to `i32`.

`c_type_to_rust()` — the C-to-Rust primitive type mapping:

```
fn c_type_to_rust(c_type: &str) -> String {
    let base = c_type.trim_end_matches('*').trim();
    match base {
        "int"           => "i32",
        "unsigned int"  => "u32",
        "long"          => "i64",
        "unsigned long" => "u64",
        "char"          => "u8",
        "float"          => "f32",
        "double"        => "f64",
        "void"           => "()",
        "size_t"        => "usize",
        _               => base, // Unknown type -> use as-is (e.g., struct names)
    }
    .to_string()
}
```

C Type	Rust Type	Notes
<code>`int`</code>	<code>`i32`</code>	32-bit signed integer
<code>`unsigned int`</code>	<code>`u32`</code>	32-bit unsigned integer
<code>`long`</code>	<code>`i64`</code>	64-bit signed integer
<code>`char`</code>	<code>`u8`</code>	C <code>`char`</code> is a byte (0-255)
<code>`float`</code>	<code>`f32`</code>	32-bit floating point
<code>`double`</code>	<code>`f64`</code>	64-bit floating point
<code>`void`</code>	<code>`()`</code>	Rust's unit type — returned by functions that return nothing
<code>`size_t`</code>	<code>`usize`</code>	Platform-native unsigned size type
Unknown	as-is	Struct/union names used directly (e.g., <code>`Node`</code> → <code>`Node`</code>)

`.trim_end_matches('*')` strips pointer stars from the type before matching.

So `"int*"`, `"int**"` all map through this function to the same `"i32"` base.

File 3: `src/module5_codegen/rust_codegen.rs`

(Synthesizing the Rust Source Code String)

Purpose: Takes the ownership maps and AST and writes Rust code. This is the "writer" that produces the candidate `.rs` file. It outputs a complete Rust module with: - Standard file header with `#![allow(...)]` attributes - Rust struct definitions from C structs - Rust enum definitions from C unions - Rust function definitions with correct ownership types

Lines 1-5: Imports

```
use std::collections::HashMap;
use crate::module2_ast::TranslationUnitAST;
use crate::data_stores::interprocedural_summaries::InterproceduralSummaries;
use super::ownership_infer::FunctionOwnershipMap;
```

Import	Note
<code>`super::ownership_infer::FunctionOwnershipMap`</code>	<code>`super`</code> refers to the PARENT module (<code>`module5_codegen`</code>). This imports from the sibling file <code>`ownership_infer.rs`</code> .

The Main Function `synthesize_rust_code()` (Lines 7-35)

```

pub fn synthesize_rust_code(
    tu_ast: &TranslationUnitAST,
    ownership_maps: &HashMap<String, FunctionOwnershipMap>,
    summaries: &InterproceduralSummaries,
) -> String {
    let mut output = String::new();

    // Header with compiler directives
    output.push_str("// Auto-generated by C2SafeRust-LLM\n");
    output.push_str("// WARNING: Review before use in production.\n\n");
    output.push_str("#![allow(unused_variables, dead_code, non_snake_case)]\n\n");

    // Generate struct definitions
    for s in &tu_ast.structs {
        output.push_str(&generate_rust_struct(s));
        output.push('\n');
    }

    // Generate enum definitions from C unions
    for u in &tu_ast.unions {
        output.push_str(&generate_rust_enum_from_union(u));
        output.push('\n');
    }

    // Generate function definitions
    for func in &tu_ast.functions {
        let func_code = generate_rust_function(func, ownership_maps, summaries);
        output.push_str(&func_code);
        output.push('\n');
    }

    output
}

```

Line	Code	Explanation
<code>`let mut output = String::new();`</code>	Empty string	We'll <code>`push_str`</code> all generated code into this.
<code>`output.push_str("...")`</code>	Append text	<code>`push_str`</code> appends a <code>`&str`</code> to the String WITHOUT allocating a new one. More efficient than <code>`output = output + "...`</code> .
<code>`#![allow(unused_variables, dead_code, non_snake_case)]`</code>	Compiler hints	Suppresses warnings that are almost always triggered in auto-generated code. <code>`#![...]`</code> (with <code>`!`</code>) is an inner attribute — applies to the ENTIRE file, not just the next item.
<code>`output.push('\n')`</code>	<code>`push`</code> vs <code>`push_str`</code>	<code>`push`</code> appends a single <code>`char`</code> . <code>`push_str`</code> appends a <code>`&str`</code> . Note the difference: <code>`'\n'`</code> (char with single quotes) vs <code>`"\n"`</code> (str with double quotes).

Generating Rust Structs from C Structs (Lines 37-56)

```
fn generate_rust_struct(struct_def: &StructAST) -> String {
    let mut s = String::new();

    s.push_str("#[derive(Debug, Clone)]\n");
    s.push_str(&format!("pub struct {} {{\n", struct_def.name));

    for (field_name, field_type) in &struct_def.fields {
        let rust_type = c_type_to_rust_simple(field_type);
        s.push_str(&format!("    pub {}: {},\n", field_name, rust_type));
    }

    s.push_str("}\n");
    s
}
```

Example transformation:

C input:

```
struct Point {
    int x;
    float y;
};
```

Generated Rust output:

```
#[derive(Debug, Clone)]
pub struct Point {
    pub x: i32,
    pub y: f32,
}
```

Step	Code	C → Rust
`s.push_str("#[derive(Debug, Clone)]\n")`	Add derives	Auto-generated structs should be printable and cloneable.
`format!("pub struct {} {{", struct_def.name)`	Struct header	`struct Point {` — note `{` in format string prints literal `{`.
`for (field_name, field_type) in &struct_def.fields`	Each field	Tuple destructuring: `(field_name, field_type)` unpacks the `(String, String)` tuple in one step.
`c_type_to_rust_simple(field_type)`	Type conversion	Maps `int` → `i32`, `float` → `f32`, etc.
`format!("    pub {}: {},\n", field_name, rust_type)`	Field line	4-space indented, `pub`, colon separator, trailing comma.

Generating Rust Enums from C Unions (Lines 58-80)

```
fn generate_rust_enum_from_union(union_def: &UnionAST) -> String {
    let mut s = String::new();
    let enum_name = capitalize(&union_def.name);

    s.push_str("#[derive(Debug, Clone)]\n");
    s.push_str(&format!("pub enum {}Enum {{\n", enum_name));

    for variant in &union_def.variants {
        let rust_type = c_type_to_rust_simple(&variant.field_type);
        let variant_name = capitalize(&variant.field_name);
        s.push_str(&format!("    {}{{}},\n", variant_name, rust_type));
    }

    s.push_str("}\n");
    s
}
```

Example transformation:

C input:

```
union Data {
    int i;
    float f;
};
```

Generated Rust output:

```
#[derive(Debug, Clone)]
pub enum DataEnum {
    I(i32),
    F(f32),
}
```

Step	How it works
<code>`capitalize(&union_def.name)`</code>	<code>`"data" → `"Data"`.</code>
<code>`format!("pub enum {}Enum", enum_name)`</code>	<code>`"DataEnum" — appending `Enum` suffix.</code>
<code>`capitalize(&variant.field_name)`</code>	<code>`"i" → `"I"`. Enum variant names must be capitalized in Rust.</code>
<code>`format!("{}", variant_name, rust_type)`</code>	<code>`" I(i32)," — tuple-style enum variant.</code>

Why tuple-style enum variants? A C union field `int i` becomes `I(i32)` — a Rust enum variant that *HOLDS* a value. When you use it: `DataEnum::I(42)` — the value is stored inside the variant. This is Rust's safe alternative to C unions: only ONE variant can be active at a time, enforced by the type system and pattern matching.

Generating Rust Functions (Lines 82-155)

```
fn generate_rust_function(
    func: &FunctionAST,
    ownership_maps: &HashMap<String, FunctionOwnershipMap>,
    summaries: &InterproceduralSummaries,
) -> String {
    let mut s = String::new();

    // Generate function signature
    let params_str = generate_rust_params(func, ownership_maps, summaries);
    let ret_type = c_type_to_rust_simple(&func.return_type);
    let ret_str = if ret_type == "()" {
        String::new()
    } else {
        format!("{}", ret_type)
    };

    s.push_str(&format!(
        "pub fn {}({}){}  {{\n",
        func.name, params_str, ret_str
    ));

    // Generate function body
    s.push_str(&generate_rust_body(func, ownership_maps, summaries));
    s.push_str("}\n");
    s
}
}
```

Return type handling:

```
let ret_str = if ret_type == "()" {
    String::new()          // void -> no return annotation at all
} else {
    format!("{}", ret_type) // e.g., " -> i32"
};
```

In Rust, `void` functions don't need `-> ()` in the signature — just omit it entirely.

So `void swap(...)` becomes `pub fn swap(...)` with no return type.

But `int foo()` becomes `pub fn foo() -> i32`.

Generating Function Parameters `generate_rust_params()` (Lines 115-142)

```

fn generate_rust_params(
    func: &FunctionAST,
    ownership_maps: &HashMap<String, FunctionOwnershipMap>,
    summaries: &InterproceduralSummaries,
) -> String {
    let map = ownership_maps.get(&func.name);
    let alias_key_prefix = format!("{}", func.name);

    func.parameters.iter()
        .map(|param| {
            // Check if this parameter is ALIASED at any call site (Idiom 1)
            let is_alias = summaries.alias_verdicts.iter()
                .any(|(key, verdicts)| {
                    key.starts_with(&alias_key_prefix) &&
                    verdicts.iter().any(|v| {
                        v.aliases && (v.arg1.contains(&param.name)
                                || v.arg2.contains(&param.name))
                    })
                })

            let rust_type = if is_alias {
                // Idiom 1: aliased mut param -> use *mut T (raw pointer, unsafe)
                format!("*mut {}", c_type_to_rust_simple(&param.c_type))
            } else if let Some(m) = map {
                // Look up inferred ownership for this parameter
                m.per_variable.iter()
                    .find(|v| v.variable_name == param.name)
                    .map(|v| v.rust_type_hint.clone())
                    .unwrap_or_else(|| format!("{}", c_type_to_rust_simple(&param.c_type)))
            } else {
                format!("{}", c_type_to_rust_simple(&param.c_type))
            };

            format!("{}", param.name, rust_type)
        })
        .collect::<Vec<String>>()
        .join(", ")
}

```

Step by step — processing each parameter:

	Code	Explanation		
	<code>`func.parameters.iter().map(\`</code>	<code>param\`</code>	<code>...)\`</code>	Transform each Rust parameter into the closure type
	Alias check	Searches <code>`summaries.alias_verdicts`</code> for any verdict that: (a) belongs to a call site of THIS function (<code>`key.starts_with(&alias_key_prefix)`</code>), AND (b) flags THIS parameter as aliased.		

	Code	Explanation		
	If aliased	Use <code>*mut T</code> (raw pointer). This is <code>unsafe</code> territory — the LLM repair agent in Module 6 will handle the <code>unsafe</code> block generation.		
	If not aliased	Look up the parameter in the <code>FunctionOwnershipMap</code> and use the inferred <code>rust_type_hint</code> .		
	Default fallback	<code>&type;</code> — assume shared borrow if no info.		
	<code>format!("{}", param.name, rust_type)</code>	Produce <code>"a: &mut; i32"</code> or <code>"buf: Box"</code> etc.		
	<code>.collect::>().join(", ")</code>	Join all parameters with <code>,</code> — <code>"a: &mut; i32, b: &mut; i32"</code> .		

Why `.collect::<Vec<String>>()` then `.join()`?

- `.map()` produces an iterator of Strings.
- `.collect()` materializes the iterator into a `Vec<String>`.
- `.join(", ")` combines the Vec into one String with `,` between elements.
- There's no way to call `.join()` directly on an iterator — it must be a slice or Vec first.

Generating Function Body `generate_rust_body()` (Lines 144-199)

```
fn generate_rust_body(
    func: &FunctionAST,
    ownership_maps: &HashMap<String, FunctionOwnershipMap>,
    _summaries: &InterproceduralSummaries,
) -> String {
    let map = ownership_maps.get(&func.name);
    let mut body = String::new();

    for line in func.body_code.lines() {
        let trimmed = line.trim();
        let transformed = transform_body_line(trimmed, map);
        if !transformed.trim().is_empty() {
            body.push_str(&format!("    {}\n", transformed));
        }
    }
    body
}
```

Code	Explanation
<code>for line in func.body_code.lines()</code>	Split the C body text into lines.
<code>transform_body_line(trimmed, map)</code>	Apply C→Rust transformations to each line (see below).
<code>format!("{}", transformed)</code>	Indent by 4 spaces (Rust convention inside function bodies).
<code>if !transformed.trim().is_empty()</code>	Skip blank lines from the transformation.

Line Transformer transform_body_line() (Lines 201-245)

```
fn transform_body_line(line: &str, map: Option<&FunctionOwnershipMap>) -> String {
    // Skip C-specific declarations that don't translate directly
    if line.starts_with("//") { return format!("{}", line); }

    // Rule: malloc -> heap allocation
    if line.contains("malloc") {
        return format!("// TODO: malloc -> Box::new(...) or vec![...]: {}\n", line);
    }

    // Rule: free() -> drop (implicit in Rust)
    if line.contains("free()") {
        return format!("// NOTE: free() is implicit in Rust via RAII: {}\n", line);
    }

    // Rule: NULL -> None
    let mut out = line.replace("NULL", "None");

    // Rule: printf -> println! (basic patterns)
    if out.contains("printf") {
        out = out.replace("printf(", "// println!");
        out = format!("{}", // TODO: convert format string\n", out);
        return out;
    }

    // Rule: Cast removal (int*) -> type inference
    out = regex_strip_casts(&out);

    // Rule: Append semicolons if missing
    if !out.ends_with(';') && !out.ends_with('{') && !out.ends_with('}')
        && !out.starts_with("//") && !out.is_empty()
    {
        out.push(';');
    }

    format!("{}", line, out)
}
```

Line-by-line transformation rules:

C Pattern	Rust Transformation	Explanation
`// comment`	Kept as-is	Comments pass through unchanged.
`malloc(n)`	`// TODO: malloc -> Box::new(...)`	`malloc` needs manual conversion. We mark it with a TODO comment for Module 6's LLM.
`free(ptr)`	`// NOTE: free() is implicit in Rust via RAII`	In Rust, `free` is automatic when a `Box` goes out of scope.
`NULL`	`None`	Rust uses `Option::None` instead of C's `NULL`.

C Pattern	Rust Transformation	Explanation
<code>`printf(...)`</code>	<code>`// println!(...) // TODO`</code>	<code>`printf`</code> format strings are different from Rust's <code>`println!`</code> format strings. Commented out for manual fixing.
C casts like <code>`(int*)`</code>	Stripped	Rust uses type inference — explicit casts from <code>`malloc`</code> aren't needed.
Missing <code>`;`</code>	Appended	C semicolons are kept. Missing ones (from broken parse) are added.

Why use *TODO* comments? The code generator marks complex transformations that need human (or LLM) review. Module 6 then takes these *TODO* markers and uses the Gemini API to generate correct replacements. This "partial translation + LLM fix" is the key innovation of the C2SafeRust-LLM pipeline.

Understanding `regex_strip_casts()`:

```
fn regex_strip_casts(code: &str) -> String {
    // Remove C-style type casts like (int*), (char*), (void*)
    let mut result = code.to_string();
    for cast in &["(int*)", "(char*)", "(void*)", "(float*)", "(double*)",
                 "(int **)", "(char **)", "(void *)", "(int *)"] {
        result = result.replace(cast, "");
    }
    result
}
```

Simple string replacement — no actual regex library needed for these specific patterns.
Removes common C casts that appear in `(int*)malloc(100)` style expressions.

The Unit Test (Lines 247-280)

```
#[test]
fn test_rust_code_synthesis() {
    // Setup: a swap function with two int** parameters
    let tu_ast = TranslationUnitAST { functions: vec![swap_func], ... };

    // Run both steps
    let ownership_maps = infer_ownership_types(&tu_ast, &models, &summaries);
    let rust_code = synthesize_rust_code(&tu_ast, &ownership_maps, &summaries);

    // Verify key characteristics
    assert!(rust_code.contains("pub fn swap"));
    assert!(rust_code.contains("&mut")); // Both params should be &mut
    assert!(rust_code.contains("#![allow("));
    assert!(!rust_code.contains("int **")); // No C types in output
}
```

Assertion	What it verifies
<code>`rust_code.contains("pub fn swap")`</code>	Function was generated with correct Rust syntax.

Assertion	What it verifies
<code>`rust_code.contains("&mut;")`</code>	Parameters correctly inferred as mutable borrows.
<code>`rust_code.contains("#![allow(")</code>	File header with suppression attributes was added.
<code>`!rust_code.contains("int **")`</code>	No raw C types remain in the output.

The Full Code Generation Flow

C Function (from AST):

```
void swap(int **a, int **b) {
    int *temp = *a;
    *a = *b;
    *b = temp;
}
```

Step 5.1 - Ownership Inference:

Function "swap":

```
"a": pattern=Write, depth=MultiLevel{2}
    -> NOT in data_dep_flags
    -> transfer_key "swap:a" NOT in ownership_summaries
    -> pattern is Write (not Allocate)
    -> RESULT: MutableBorrow -> rust_type_hint = "&mut i32"
"b": same -> "&mut i32"
"temp": pattern=Read, depth=SingleLevel
    -> RESULT: SharedBorrow -> "&i32"
```

Step 5.2 - Code Synthesis:

generate_rust_function("swap"):

```
params: "a: &mut i32, b: &mut i32"
ret_type: "()" -> no return annotation
signature: "pub fn swap(a: &mut i32, b: &mut i32) {"
body:
    "int *temp = *a;" -> "let temp = *a;" (type inferred)
    "*a = *b;" -> "*a = *b;" (dereference assignment)
    "*b = temp;" -> "*b = temp;"
```

Generated Rust:

```
pub fn swap(a: &mut i32, b: &mut i32) {
    let temp = *a;
    *a = *b;
    *b = temp;
}
```

Key Rust Concepts Introduced in This Module

	Meaning			
<code>write_code()</code>	Write a String to a file atomically.			
	Safe even for paths with non-UTF-8 chars.			
<code>String::new()</code>	Empty String — more efficient than <code>String::new()</code> .			
	Append a string slice to a String.			
	Append a single character.			
<code>#[warn]</code>	Inner attribute — applies to whole file, suppresses warnings.			
	<code>Vec::collect()</code>	Parameter generation	Functional transform: iterator → map each element → collect to Vec.	
	Combine Vec with a separator.			
<code>use crate::</code>	Access parent module's items from a sibling file.			
<code>for</code> in	Unpack a <code>(String, String)</code> tuple in a <code>for</code> loop.			
	<code>unwrap_or_else()</code>	<code>.unwrap_or_else()</code>	<code>\</code>	<code>"i32".to_string()</code>
	<code>cond()</code>	<code>.any()</code>	<code>v\</code>	<code>v.aliases &&</code>
	In <code>format!</code> , <code>{ }</code> is an escaped literal <code>{ }</code> .			
	Check the last character. Returns <code>bool</code> .			

Summary

Module 5 is the **transformation engine** — it converts analysis results into actual Rust code:

5.1 Ownership Inference (`ownership_infer.rs`):

- For every variable in every function, applies a 5-rule priority decision tree
- Consults D2 first (data-dep flags, transfer summaries) then D1 (memory patterns, pointer depth)
- Maps each variable to `OwnershipKind`: `UniqueHeapOwner`, `MutableBorrow`, `SharedBorrow`, `HeapArray`, or `ConservativeFallback`
- Converts `OwnershipKind` to Rust type strings: `Box<T>`, `&mut T`, `&T`, `Vec<T>`, `Option<Box<T>>`

5.2 Rust Code Synthesis (`rust_codegen.rs`):

- Generates file header with `#![allow(...)]` directives
- Translates C `struct` → Rust `pub struct` with `#[derive(Debug, Clone)]`
- Translates C `union` → Rust `pub enum` with capitalized variant names
- Generates Rust function signatures using inferred ownership types
- Transforms function body lines: `NULL` → `None`, marks `malloc/free` as TODOs for Module 6

- Strips C-style casts, ensures semicolons present
 - Writes the complete Rust source to `output.rs` for Module 6's `rustc` validation loop
-

Document generated for C2SafeRust-LLM Mini Project S3

Module 5: Ownership Inference & Rust Code Generation — Complete Line-by-Line Explanation